

Labdagép (Ball Machine)

Madina feltalált egy labdagépet, hogy szórakoztassa az IOI résztvevőit. A gép belső felépítése a következő:

- A gép N **csomópontból** áll, amelyek 0 -tól $N - 1$ -ig indexeltek. Az $N - 1$ -es csomópontot **gyökérnek** nevezzük.
- Minden u csomópontra, ahol $0 \leq u < N - 1$, a u csomópont **szülője** egy $P[u]$ csomópont, ahol $P[u] > u$, és az u csomópont a $P[u]$ **gyereke**. A gyökérnek nincs szülője. A gyerek nélküli csomópontokat **levélnek** nevezzük.

Nem ismered N értékét, és semelyik u csomópont $P[u]$ szülőjét sem. Ehelyett Madina megmondja M -et, a levelek számát, és hogy a levelek indexei $0, 1, \dots, M - 1$. A feladatod az, hogy a gép működtetésével meghatározd a belső szerkezetét.

A gép minden csomópontja legfeljebb egy **labdát** tud tartani, és minden labdának van egy **értéke**, amely egy nemnegatív egész szám. Azt mondjuk, hogy egy csomópont értéke x , ha egy x értékű labdát tart. Egy csomópont **foglalt**, ha tartalmaz egy labdát; egyébként **szabad**. Kezdetben minden csomópont szabad.

Kétféle műveletet végezhetsz:

- insert(U, X)**: megpróbál beszúrni egy új, X értékű labdát az U levélre.
 - Ha az U levél foglalt, a művelet `false` értéket ad vissza a labda behelyezése nélkül.
 - Ha az U levél szabad, a labda az U csomópontba kerül. Ezután a labda ismételten az aktuális csomópont szülőjéhez mozog, amíg az a szülő szabad. A mozgás megáll, amikor a labda eléri a gyökeret vagy egy olyan csomópontot, amelynek a szülője foglalt. A művelet ezután `true` értéket ad vissza.
- collect()**: ha a gyökér szabad (azaz a gép üres), akkor a `collect` egy üres tömböt ad vissza. Ellenkező esetben az alább ismertetett rekurzív `traverse` eljárás összegyűjti a gépben lévő összes labda értékét egy S tömbbe. Kezdetben az S tömb üres, és a `traverse(N - 1)` hívást hajtjuk végre.

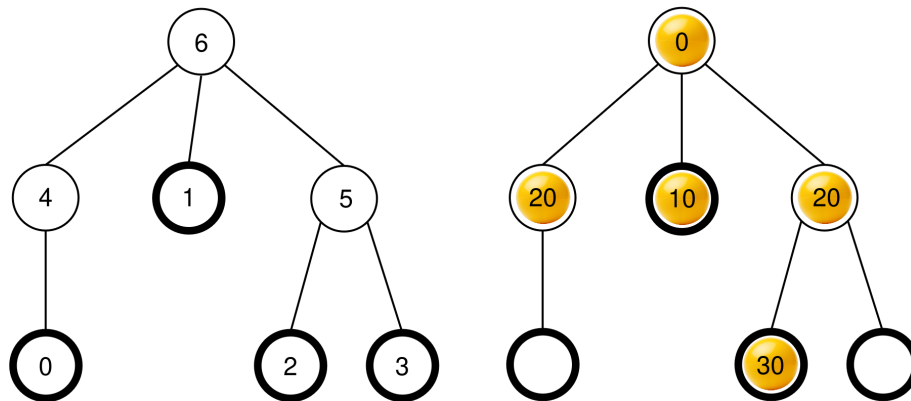
```

traverse(u) :
    az u csomópontban lévő labda értékét tegyük az S tömb végére
    legyen c[u] az u csomópont foglalt gyerekeinek listája
    rendezzük c[u]-t nemcsökkenő sorrendbe az adott csomópontokban
    lévő labdák értékei alapján (ha több csomópontban azonos
    érték van, akkor bármely sorrendben lehetnek)
    minden v csomópontra c[u]-ban:
        traverse(v)

```

Miután ez az eljárás befejeződött, **az összes labdát eltávolítjuk** a gépből, és a `collect` művelet az S tömböt adja vissza.

Például vegyünk egy $N = 7$ csomóponttal rendelkező gépet, amelynek a szülő tömbje $P = [4, 6, 5, 5, 6, 6]$. A bal oldali kép az üres gépet mutatja csomópontindexekkel, míg a jobb oldali kép a labdák egy lehetséges elhelyezkedését mutatja be egy bizonyos `insert` műveletsorozat után (ez a sorozat a Példa részben található).



Amikor a `collect()` függvényt meghívjuk, a gyökér (a 6-os csomópont) foglalt, így az $S = []$ inicializálás történik, és a `traverse(6)` függvényhívás hajtódik végre.

traverse(6): A 6-os csomópont értéke 0; ha ezt hozzáfűzzük S -hez, akkor $S = [0]$ lesz. A 6-os csomópont gyerekei 4, 1 és 5, amelyek mindegyike foglalt. Értékük rendre 20, 10 és 20. A nemcsökkenő sorrendbe rendezés után $c[6] = [1, 5, 4]$ lesz (megjegyezzük, hogy $c[6] = [1, 4, 5]$ is érvényes lenne, mivel a 4-es és 5-ös csomópontok értéke megegyezik). Az eljárás ezután `traverse` függvényt hívja meg $c[6]$ minden csomópontján ebben a sorrendben.

- **traverse(1):** Az 1-es csomópont értéke 10; ha ezt hozzáfűzzük S -hez, akkor $S = [0, 10]$ lesz. Az 1-es csomópontnak nincsenek foglalt gyerekei, így a hívás véget ér.
- **traverse(5):** Az 5-ös csomópont értéke 20; ha ezt hozzáfűzzük az S tömböz, akkor $S = [0, 10, 20]$ lesz. Az 5-ös csomópontnak egy foglalt gyereke van, a 2-es csomópont, tehát $c[5] = [2]$, és az eljárás meghívja `traverse(2)`-t.

- **traverse(2)**: A 2-es csomópont értéke 30; ha ezt hozzáfűzzük S -hez, akkor $S = [0, 10, 20, 30]$ lesz. A 2-es csomópontnak nincsenek foglalt gyerekei, így a hívás véget ér.
- A végrehajtás visszatér a `traverse(5)` függvényhez, amely feldolgozta $c[5]$ összes gyerekeit, így a hívása véget ér.
- **traverse(4)**: A 4-es csomópont értéke 20; ha ezt hozzáfűzzük S -hez, akkor $S = [0, 10, 20, 30, 20]$ lesz. A 4-es csomópontnak nincsenek foglalt gyerekei, így a hívás véget ér.

Minden hívás befejeződött, és nincsenek további feldolgozandó csomópontok. Végül minden labdát kiveszünk a gépből, és `collect` függvény az $S = [0, 10, 20, 30, 20]$ tömböt adja vissza.

A feladatod a csomópontok N számának meghatározása, és egy $R = [R[0], R[1], \dots, R[N-2]]$ szülőkből álló tömb megadása, amely leírja a gép szerkezetét, egy potenciálisan eltérő címkézéssel a nem levél és nem gyökér csomópontokon. Formálisan egy R tömböt akkor tekintünk helyesnek, ha a gép minden u csomópontjához hozzárendelhetők egymástól különböző $L[u]$ címkék $0 \leq L[u] < N$ értékkel úgy, hogy:

- $L[u] = u$ minden $0 \leq u < M$ és $u = N - 1$ esetén, és
- $L[P[u]] = R[L[u]]$ minden $0 \leq u < N - 1$ esetén.

Nem feltétel, hogy $R[u] > u$. A gépnek **üresnek** kell lennie, amikor a megoldást közlöd.

Legyen K a végrehajtott `collect` műveletek száma, B pedig az összes `insert` hívás során bármely labda maximális értéke. Legyen $C = K + B$. Ekkor C **nem lehet több, mint 1000**. Egyes részfeladatokban a pontszám C értékétől függ.

Megvalósítás

A következő függvényt kell implementálnod.

```
std::vector<int> find_structure(int M)
```

- M : a gépben lévő levelek száma.
- A függvény minden tesztesetre pontosan egyszer lesz meghívva.
- Az függvénynek egy $N - 1$ elemű $R = [R[0], R[1], \dots, R[N-2]]$ tömböt kell visszaadnia, amely a szülők helyes tömbje.

A géppel való interakcióhoz a függvényed a következő két függvényt hívhatja meg.

Az első függvény:

```
bool insert(int U, int X)
```

- U : annak a levélnek az indexe, ahová a labdát be szeretnénk helyezni. A $0 \leq U < M$ feltételnek teljesülnie kell.
- X : egy egész szám, a labda értéke. A $0 \leq X \leq 1000$ feltételnek teljesülnie kell.
- Ez a függvény `true` értéket ad vissza, ha a labda sikeresen behelyezésre került, illetve `false` értéket, ha az U levél már foglalt volt.
- Ez a függvény legfeljebb 500 000 alkalommal hívható meg.

A második függvény:

```
std::vector<int> collect()
```

- Begyűjti az összes behelyezett labda értékét, kiüríti a gépet, és visszaadja a begyűjtött S tömböt.

Ha a program végrehajtása során bármikor a C értéke meghaladja az 1000-et, a megoldás az `Output isn't correct: Too many resources used` visszajelzést fogja kapni.

A gép szerkezete rögzítve van a `find_structure` meghívása előtt. Az értékelő **determinisztikus** abban az értelemben, hogyha kétszer futtatjuk le, és mindkét futtatás ugyanazt a műveletsorozatot hajtja végre, akkor a `collect` hívások ugyanazokat a tömböket fogják visszaadni.

Korlátok

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Részfeladatok

Részfeladat	Pontszám	További megkötések
1	5	A gyökérnek pontosan M gyereke van.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Nincsenek további megkötések.

A 3. és 4. részfeladatban a pontszám a C értékétől függ az alábbiak szerint.

3. részfeladat (25 pont)

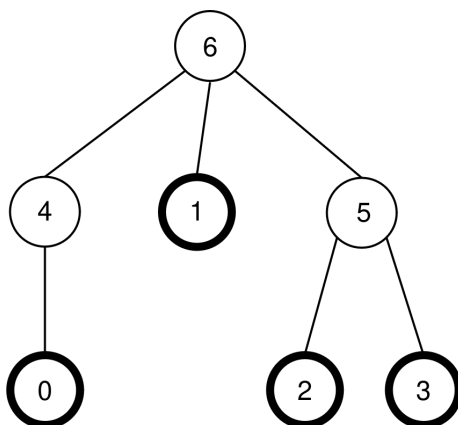
Korlátok	Pontszám
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

4. részfeladat (60 pont)

Korlátok	Pontszám
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Példa

Tekintsük ugyanazt a gépet, mint a feladatleírásban, tehát $N = 7$, $M = 4$, és $P = [4, 6, 5, 5, 6, 6]$. A gépet ismét a következő ábra mutatja:



Az értékelő a következő hívást hajtja végre:

```
find_structure(4)
```

A függvény végrehajthatja a következő hívássorozatot:

1. **insert(0, 0)**: A 0-ás levél szabad, így egy 0 értékű labdát helyezünk a 0-ás levélre. $P[0] = 4$ csomópont szabad, így a labda a 4-es csomópontba mozog. A $P[4] = 6$ -os

csomópont szabad, így a labda a 6-os csomópontba mozog. Mivel a 6-os csomópont a gyökér, a mozgás megáll. A hívás `true` értéket ad vissza.

2. `insert(3, 20)`: A 3-as levél szabad, így egy 20 értékű labdát helyezünk a 3-as levélre. A $P[3] = 5$ -ös csomópont szabad, így a labda az 5-ös csomópontba mozog. Mivel a $P[5] = 6$ -os csomópont foglalt, a mozgás megáll. A hívás `true` értéket ad vissza.

3. `insert(1, 10)`: Az 1-es levél szabad, így egy 10 értékű labdát helyezünk az 1-es levélre. Mivel a $P[1] = 6$ -os csomópont foglalt, a labda az 1-es csomópontban marad. A hívás `true` értéket ad vissza.

4. `insert(2, 30)`: A 2-es levél szabad, így egy 30 értékű labdát helyezünk a 2-es levélre. Mivel a $P[2] = 5$ -ös csomópont foglalt, a labda a 2-es csomópontban marad. A hívás `true` értéket ad vissza.

5. `insert(1, 25)`: Az 1-es levél foglalt, így a labda nem kerül az 1-es levélre. A hívás `false` értéket ad vissza.

6. `insert(0, 20)`: A 0-ás levél szabad, így egy 20 értékű labdát helyezünk a 0-ás levélre. A $P[0] = 4$ -es csomópont szabad, így a labda a 4-es csomópontba mozog. Mivel a $P[4] = 6$ -os csomópont foglalt, a mozgás megáll. A hívás `true` értéket ad vissza.

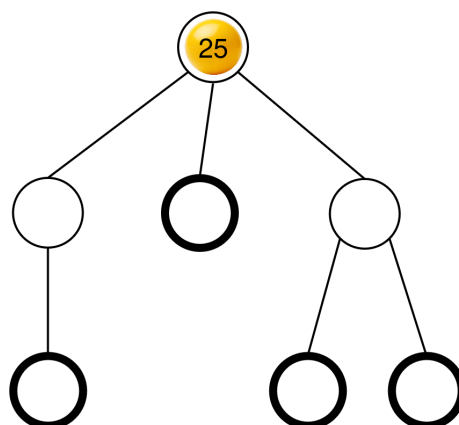
A labdák végső elhelyezkedése már szerepelt a feladatléírásban.

Ezután a függvény meghívhatja a következőket:

```
collect()
```

A hívás végrehajtását már ismertettük a feladatléírásban, a hívás az $S = [0, 10, 20, 30, 20]$ tömböt adja vissza. Ezután a gép ismét üres.

Ezután a függvény meghívhatja a `insert(2, 25)` függvényt. A 2-es levél szabad, így egy 25 értékű labdát helyezünk a 2-es levélre. Ez a labda ezután az 5-ös csomópontba, majd a 6-os csomópontba mozog, ahol megáll. A hívás `true` értéket ad vissza. A labdák létrejött elhelyezkedése a következő ábrán látható.



Végül a függvény meghívhatja a `collect()` függvényt, amely az $S = [25]$ tömböt adja vissza és kiüríti a gépet.

A `find_structure` két helyes szülő tömböt adhat vissza: $R = P = [4, 6, 5, 5, 6, 6]$ (ami az $L = [0, 1, 2, 3, 4, 5, 6]$ címkézésnek felel meg), és $R = [5, 6, 4, 4, 6, 6]$ (ami az $L = [0, 1, 2, 3, 5, 4, 6]$ címkézésnek felel meg). Ebben a példában $K = 2$ és $B = 30$, tehát $C = 32$.

Mintaértékelő

Bemeneti formátum:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Kimeneti formátum:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Itt Q a `find_structure` által visszaadott R tömb hossza.